Robot-Agnostic Constrained Inverse Kinematics

Take-home engineering assignment

> **Suggested timebox:** 3–4 hours
> **Environment:** Simulation only; no physical robot is required
> **AI assistance:** Allowed and encouraged. Please briefly describe how you used it.

Context

We want to command different robots in Cartesian space without depending exclusively on the inverse-kinematics implementation embedded in a robot vendor's SDK.

The system should accept a desired end-effector pose and compute an appropriate joint configuration while respecting robot constraints. It should also remain stable and predictable when the requested pose cannot be reached.

Your task

Design and implement a proof-of-concept constrained inverse-kinematics system. The system should expose an interface broadly similar to the following; the exact API is up to you:

```
text solve( robot_model, target_pose, current_joint_position, constraints
) -> solution -
```

Core requirements

Your prototype should:

- Represent a robot independently of a particular vendor SDK.
- Accept a Cartesian end-effector target.
- Compute a joint-space solution using your own solver logic or a general numerical optimization library.
- Respect joint-position limits.
- Prefer solutions that are reasonably close to the current joint configuration.
- Return useful status information, such as success, approximate solution, invalid input, or non-convergence.
- Produce a sensible and well-defined result for an unreachable target.

You should define what "sensible" means. For example, the solver might return the closest pose it found, avoid unstable joint jumps, and report the remaining Cartesian error.

> **Scope note:** A simple planar arm is acceptable. Full inverse dynamics, collision checking, and real-time execution are not required. If you believe inverse dynamics belongs in a different layer, explain how you would separate those responsibilities.

Required demonstrations

Demonstrate the solver on at least:

1. A reachable target.
2. An unreachable target.
3. A target close to a joint limit.

If practical within the timebox, show that the same solver interface works with two different robot configurations. This is optional; a clear robot-agnostic design is more important than a lengthy implementation.

Design discussion

In a short README or design note, explain:

- Your solver approach and why you chose it.
- How robot-specific kinematics are separated from solver logic.
- How you detect or handle unreachable targets.
- How you balance pose accuracy against joint motion and constraints.
- Potential issues near singularities.
- What would be required to make the system safe and useful on a physical robot.
- How you would extend it to support velocity limits, collision constraints, redundancy resolution, or dynamics-aware optimization.

Deliverables

- Runnable source code.
- A short README with setup and usage instructions.
- A few tests or demonstrations covering the scenarios above.
- A concise discussion of design choices, limitations, and next steps.
- A brief note describing any AI tools used and how you validated their output.

Production-grade code, ROS integration, visualization, and a polished user interface are not expected. Prioritize a clear, working core and explicit engineering decisions.

Evaluation criteria

Area	What we look for
Working proof of concept	Correct core behavior and convincing demonstrations
System design	Clean separation between the robot model, solver, and constraints
Failure behavior	Predictable handling of unreachable targets, limits, and non-convergence
Technical reasoning	Awareness of numerical behavior, singularities, safety, and tradeoffs
Clarity	Readable code, concise documentation, and reproducible execution

Candidates are not penalized for consciously omitting features outside the timebox, provided the limitations are identified and the proposed next steps are well reasoned.